

GitHub Copilot Deep Dive

Document History

Ver. Rel. No.	Release Date	Prepared. By	Reviewed By	Approved By	Remarks/Revision Details

Course Title: GitHub Copilot Deep Dive

1.Course Summary:

This program enables developers to seamlessly integrate GitHub Copilot into their daily workflows, enhancing productivity and code quality. Participants will learn to apply advanced prompting strategies, leverage Copilot Chat, Edit, and Agent features, and automate CI/CD pipelines. Hands-on modules cover code refactoring, test automation, bug detection, and secure coding practices. Developers will build end-to-end applications while enforcing best practices and design patterns suggested by Copilot. By mastering GenAI tools, participants gain a competitive edge in modern software development.

2. Pre-Requisite -

- Be able to write code in any modern programming language, such as Java, Python, JavaScript, or C# (Program will use Java).
- Possess necessary foundation skills in programming logic and techniques, basic data structures, and algorithms.
- Have experience in developing backend services (RESTful APIs).
- Have hands-on experience using Git, Github, and Github Actions.

3. Target Audience; All associates

4. Hardware & Network Requirements; NA

Software Requirements -

- **GitHub Copilot**
 - Subscription: GitHub Copilot Individual.
 - Install VS Code.
- **Install extensions:**
 - GitHub Copilot
 - GitHub Copilot Chat
 - C/C++ Extension Pack
- **GitHub & CI/CD**
 - Git installed (to push/pull code)
 - GitHub Actions (built-in, no extra setup)
- **Cloud Deployment (Capstone Project)**
 - AWS Free Tier (EC2 instance to run C program)

5. Learning Objectives:

- Use Copilot in their day-to-day programming tasks.
- Augment Copilot capabilities with advanced prompting strategies.
- Use Copilot advanced capabilities for various software development use cases. This includes using Copilot Chat, Edit, and Agent features.
- Leverage Copilot for code refactoring.
- Automate CI/CD pipelines, Github Actions configuration generation with Copilot.
- Perform test automation with Copilot.
- Detect and correct bugs in code. This includes using Copilot to identify potential bugs and suggest fixes.
- Be more productive by focusing on core business requirements rather than boilerplate code, leading to faster development cycles.
- Enforce best practices, coding conventions, and coding patterns as suggested by Copilot, which can help improve the readability and maintainability of code.
- Develop an end-to-end application using the capabilities of Copilot.
- Gain a competitive advantage over those who are not using Gen AI tools and technology.

6. Course Content with Lab/Hands-on Activity (day wise break-up):

Getting Started with GitHub Copilot – 3 Hours

- Introduction to GitHub Copilot- - Key Features and Capabilities.
- A brief history and development of GitHub Copilot.
- Basic concepts and principles behind GitHub Copilot's code suggestion.
- Identify the range of languages and frameworks support.
- Learn about the supported IDEs and Editors.
- Explore real-world applications and use-cases of GitHub Copilot in software development.
- Discuss real world customer success stories/case studies on GitHub Copilot.
- Installation and Environment Setup
- Make use of GitHub Copilot Code Completion capabilities (inline completion, navigating between suggestions, prompting through comments)
- Demonstrate code generation examples (programming constructs and design patterns).
- Explore Copilot's range of suggestions from snippets, functions, to whole blocks of code.
- Demonstrate how to accept, modify, or reject GitHub Copilot suggestions.
- Experiment with GitHub Copilot prompts to generate better code suggestions.
 - **Enhanced Prompting Strategies** Deeper patterns: In-Code Exemplars (Few-Shot Learning within context)
- Instructional Scaffolding (Step-by-Step Guidance))
 - Contextual Priming with constraints and roles
 - Pattern Seeding and Iterative Refinement (start from broader prompt to incrementally add specific constraints.
 - Follow-up vs new chat prompts
 - Context-added prompting for better code suggestions.
 - Use negative prompts to exclude unwanted suggestions.
 - Chain prompts for complex code generation scenarios.
- Developing a sample application in the core language: Generate code while applying Design Patterns (Singleton/Builder) to complete the application.
- Discuss training data, model accuracy, and limitations of GitHub Copilot.
- Identify future trends in GenAI that could impact AI tools for code generation.
- Explore potential advancements in GitHub Copilot features and capabilities.
- **Comparative Analysis of AI Coding Companions**
 - GitHub Copilot vs Amazon CodeWhisperer vs Tabnine feature analysis
 - Strengths and limitations of each platform
- Highlight ethical, data privacy considerations and the role of Gen AI in the future of coding.

Apply GitHub Copilot Capabilities – 3 Hour

- Explore the capabilities of GitHub Copilot Chat
- Use different slash commands
- Attach images to chat prompts
- Leverage voice chat
- Reference file, method, class in Chat
- Reference entire solution
- Review the sources used by Copilot Chat
- Organize chats into threads
- Chat with extensions
- Apply Copilot Chat best practices
- Make use of Inline Editor Chat and Quick Chat.
- Learn about supported models and switching between them.
- Demonstrate GitHub Copilot in the command line
- Perform code debugging with GitHub Copilot
- Understand context-aware suggestions and the aspects that GitHub Copilot uses to make context-aware suggestions.
- Demonstrate configuring GitHub Copilot settings on GitHub.com and IDE/Editor.
- Use Copilot Edits for multi-file editing
- When to use Copilot Edits vs. Copilot Chat?

- Experiment with Copilot Agent mode.
- Guide GitHub Copilot with custom copilot instructions.
- Roll out Capstone project Case Studies.

Code Refactoring with GitHub Copilot – 1.5 Hours

- Identify code smells and anti-patterns using Copilot.
- Demonstrate existing inefficient, vulnerable, and redundant code and then use GitHub Copilot to identify and refactor them.
- Large-scale refactoring strategies with Copilot assistance.
- Legacy code modernization using Copilot.
- Refactoring Patterns and Techniques: Extract method, class, and interface refactoring.
- Design pattern implementation through refactoring.
- Code consolidation and duplication removal.
- Security-focused Refactoring: Identifying and fixing security vulnerabilities.
- Secure coding pattern implementation.
- Input validation and sanitization improvements.
- Refactoring demo with real codebases - Before/after code quality metrics analysis.
- Team refactoring best practices.

Make Code Production Ready – 3 Hours

CI/CD and Automation Integration

- Copilot assistance in CI/CD pipeline configuration – JSON and YAML configurations.
- Leverage Copilot to handle environment variables and Secrets in CI/CD automation workflows.
- Use Copilot for GitHub Actions workflow generation
- Get scripting assistance for automation tasks within Workflows (Bash, Python, PowerShell)
- Generate code for conditional Step/Job Execution in workflows
- Generate cron schedules in Github action workflow
- Get Copilot suggestions for relevant GitHub Marketplace actions
- Refactor and Optimize Existing Workflows

Test automation with Copilot

- Prompt Copilot for suggestions on test cases
- Generate test cases and assertions (unit, integration, frontend e2e)
- Generate mock setups for dependencies
- Cover edge, negative cases and improve test coverage
- Refactor tests for readability and maintainability.
- Generate test docs/comments

Code Documentation

- Class level/method level code description/comments
- Inline code documentation– List Steps
- API endpoint documentation
- Code review
- Bug detection and advanced debugging.
- Code optimization
- Make code cleaner
- Make code concise and robust
- **Demo: Building an end-to-end solution using GitHub Copilot.**

Advanced Topics – 1.5 Hours

- Understand security compliance for organizations.

- Provide an overview of the importance of security compliance in software development.
- Emphasize the importance of understanding established security policies and secure coding guidelines of the organization.
- Understand risks (Code security, Data privacy, compliance risks)
- Discuss integrating GitHub Copilot into a secure Software Development Lifecycle (SDLC)
- Learn best practices to secure use of GitHub Copilot (Review and testing, security specific GitHub Copilot configuration and customization).
- Consolidate tips and tricks and best practices for using GitHub Copilot.
- Discuss potential issues/pitfalls and how to avoid them.
- Discuss potential issues like over-reliance on GitHub Copilot, misunderstanding AI-generated code, etc.
- Identify strategies for critically evaluating Copilot's suggestions.
- Explore the legal (copyright) implications of using and distributing code generated by Copilot, and what responsibilities do developers have in this context?
- Explore ethical and bias issues and their counter measures.
- Explore potential advancements in Copilot's features and capabilities.

Capstone Project: Build an MVP using GitHub Copilot – 4 hours

Build an application based on rolled out case study, present with insights on Copilot's impact on workflow, productivity, challenges, and areas for exploration.

Participant to form groups to:

1. Build an MVP using GitHub Copilot as a coding companion.
2. Present the solution.
3. Share observations on parameters such as productivity gains, challenges encountered, learning takeaways, adoption plans, and approaches to leverage GitHub Copilot in development workflows.

Use Cases

Vehicle Telemetry Visualization: A system that ingests vehicle telemetry (speed, battery level, GPS) and visualizes it for fleet operators.

Digital Twin Assembly-Line Simulation: Create a digital twin of an assembly line that simulates throughput under varying parameters (cycle times, downtime, batch sizes).

Shop-Floor Resource Allocation: A system that allows manufacturing supervisors to assign operators, machines, and materials to work orders and adjust allocations on the fly to minimize idle time.

7.Course Structure:

Activity	Indicative Number of Hours
Pre-Read Hours	
Teaching Hours	12 Hours
Hands on Sessions Hours	
Assignments & Tutorial Hours	
Mock Project Hours	4 hours
Lab/Tool Access post Training	

8.Course Evaluation:

Method of Assessment	Yes/No	Weightage
Pre-Assessment	No	
Mid-Assessment	No	
Post-Assessment	Yes	
Project Work	Yes	

9.Course Resources:

- a.
- b.
- c.
- d.
- e.
- f.

10. Recommended Reading Links:

- a.
- b.
- c.
- d.
- e.
- f.

11. Course Owner (s):

Employee Name	Employee Mail ID	Business Unit